Studentessa	Corso	Attività svolta
Martina Foresta	Cybersecurity	Analisi e mitigazione di un Buffer Overflow

Analisi e mitigazione di un Buffer Overflow

1 Obiettivo

L'esercitazione ha mostrato come un programma C possa gestire in modo non sicuro un input più lungo dello spazio disponibile in memoria. È stato prima osservato il comportamento del programma vulnerabile, poi è stata verificata una modifica solo parziale della dimensione del buffer e infine è stato limitato il numero di caratteri letti.

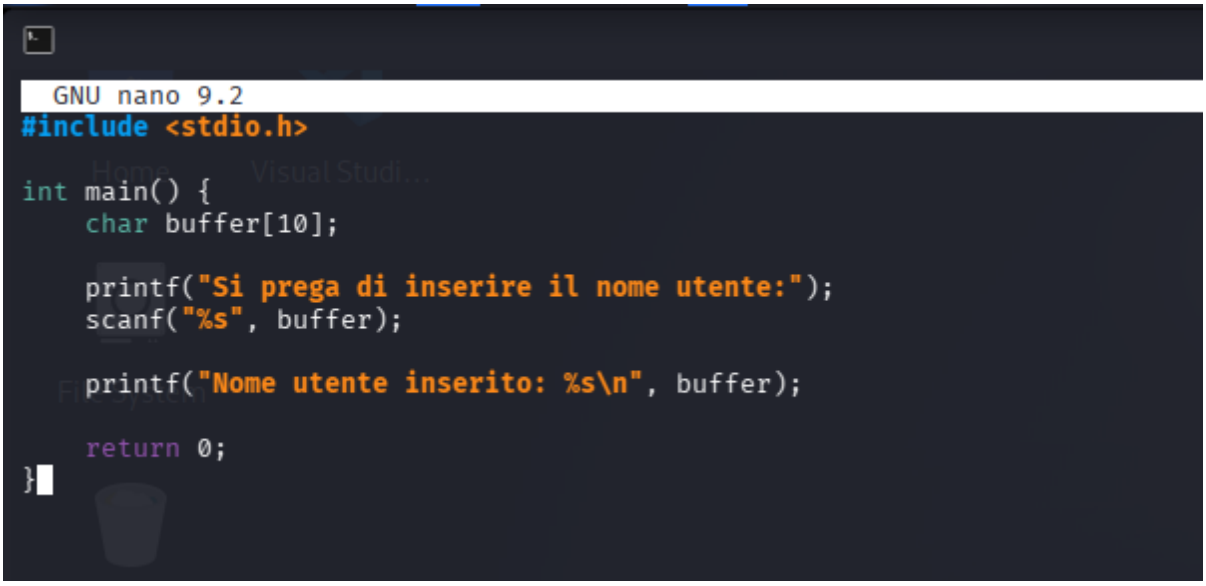
2 Preparazione dell'ambiente

Nel terminale di Kali è stato eseguito `cd Desktop`. Il comando `cd` cambia la cartella di lavoro del terminale; in questo modo il file viene creato e compilato sul Desktop.



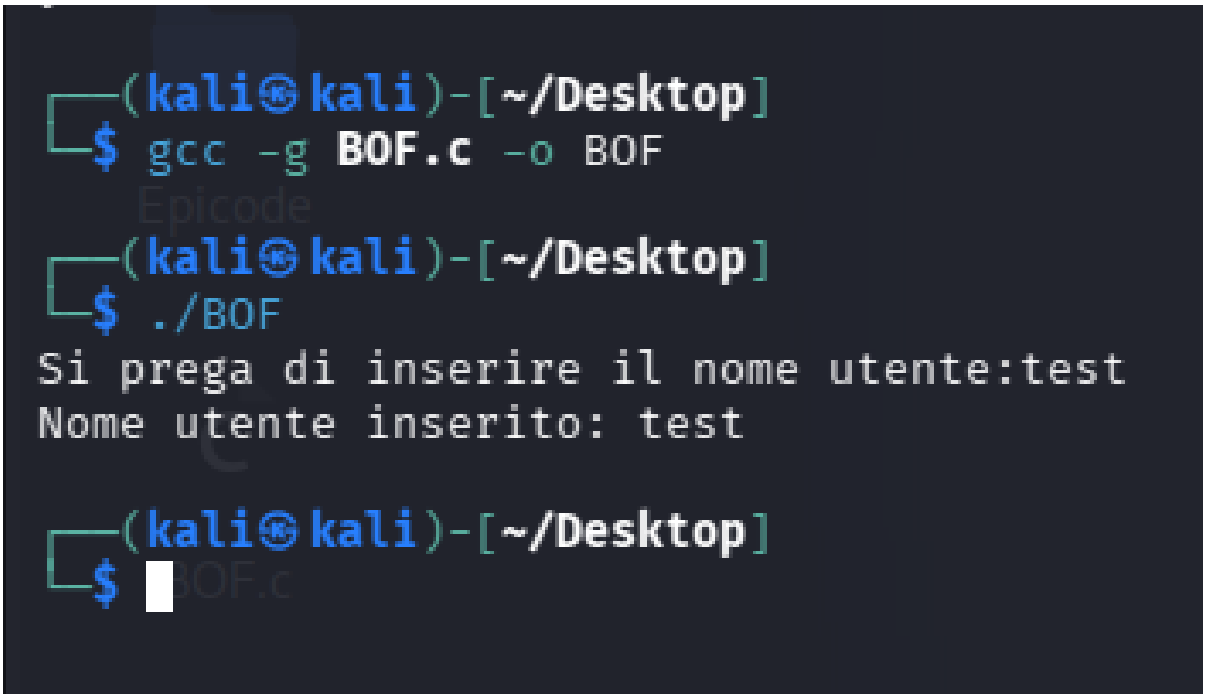
3 Programma iniziale

È stato creato `BOF.c` con l'editor `nano`. Il programma dichiara `char buffer[10]`, quindi riserva uno spazio limitato. La funzione `scanf` con `%s` legge una stringa, ma nella forma iniziale non riceve un limite massimo di caratteri.



4 Compilazione e input normale

Il comando `gcc -g BOF.c -o BOF` compila il file C e crea l'eseguibile `BOF`. L'opzione `-g` aggiunge informazioni utili per eventuali verifiche. Il comando `./BOF` avvia l'eseguibile presente nella cartella corrente. Con un nome breve, il programma ha stampato correttamente il valore ricevuto.



5 Errore di segmentazione

Inserendo un nome molto più lungo dei 10 caratteri disponibili, l'output ha mostrato `zsh: segmentation fault ./BOF`. Il programma si è interrotto dopo aver tentato di gestire l'input oltre lo spazio previsto. Questo è il comportamento osservato durante la prova del buffer overflow.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./BOF
Si prega di inserire il nome utente:qwertyuiopasdfghjklzxcvbnm1234567890
Nome utente inserito: qwertyuiopasdfghjklzxcvbnm1234567890
zsh: segmentation fault  ./BOF

┌──(kali㉿kali)-[~/Desktop]
└─$
```

6 Aumento del buffer

La dichiarazione è stata modificata da `char buffer[10]` a `char buffer[30]`. La modifica riserva più spazio, ma non cambia il fatto che `%s` non imponga un limite alla lettura.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ nano BOF.c

┌──(kali㉿kali)-[~/Desktop]
└─$ cat BOF.c
#include <stdio.h>

int main() {
    char buffer[30];

    printf("Si prega di inserire il nome utente:");
    scanf("%s", buffer);

    printf("Nome utente inserito: %s\n", buffer);

    return 0;
}
```

Dopo la ricompilazione, un input superiore a 30 caratteri non ha prodotto `segmentation fault` nella prova raccolta. Questo risultato non dimostra però che il programma sia sicuro: un input ancora più lungo potrebbe comunque superare il buffer.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ gcc -g BOF.c -o BOF

┌──(kali㉿kali)-[~/Desktop]
└─$ ./BOF
Si prega di inserire il nome utente:qwertyuiopasdfghjklzxcvbnm1234567890
Nome utente inserito: qwertyuiopasdfghjklzxcvbnm1234567890

┌──(kali㉿kali)-[~/Desktop]
└─$
```

7 Limitazione dell'input

L'istruzione è stata modificata da `scanf("%s", buffer);` a `scanf("%29s", buffer);`. Con `buffer[30]`, il valore 29 limita i caratteri letti e lascia un posto per `\0`, il carattere finale che indica la fine della stringa in C.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ nano BOF.c

┌──(kali㉿kali)-[~/Desktop]
└─$ cat BOF.c
#include <stdio.h>

int main() {
    char buffer[30];

    printf("Si prega di inserire il nome utente:");
    scanf("%29s", buffer);

    printf("Nome utente inserito: %s\n", buffer);

    return 0;
}
```

Con la nuova versione, un input molto lungo è stato limitato alla parte ammessa e non ha prodotto un errore di segmentazione. Questa è la correzione verificata nell'esercitazione.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ gcc -g BOF.c -o BOF

┌──(kali㉿kali)-[~/Desktop]
└─$ ./BOF
Si prega di inserire il nome utente:qwertyuiopasdfghjklzxcvbnm1234567890
Nome utente inserito: qwertyuiopasdfghjklzxcvbnm123

┌──(kali㉿kali)-[~/Desktop]
└─$
```

8 Conclusioni

Il programma iniziale ha prodotto un `segmentation fault` con un input troppo lungo. Aumentare il buffer a 30 caratteri ha evitato il crash osservato nella prova successiva, ma non limitava la lettura. La specifica `%29s` ha imposto invece un limite coerente con `buffer[30]` e ha impedito che l'input lungo venisse copiato oltre lo spazio previsto.